



LOVELY PROFESSIONAL UNIVERSITY
PHAGWARA, PUNJAB

TRAINING REPORT

**SMARTCOURIER DELIVERY MANAGEMENT
SYSTEM**

Java Full Stack Training Project

Submitted in partial fulfillment of the requirements for the award of degree of
B. Tech Computer Science Engineering

Training Organization: Capgemini

Training Duration: 18 December 2025 to 21 May 2026

SUBMITTED BY	SUBMITTED TO
Name of Student: <u>Sanvi Ojha</u> Registration Number: <u>12212400</u>	Name of Supervisor: Capgemini Designation: Company
Signature of Student: <u>Sanvi Ojha</u>	

Student Declaration

I hereby declare that this training report titled SmartCourier Delivery Management System is an authentic record of the project work carried out by me during the Java Full Stack Intern training program at Capgemini from 18 December 2025 to 21 May 2026. The work described in this report is based on guided learning, implementation practice, testing, debugging, source-code study, and documentation completed during the training period.

SmartCourier is an academic training project prepared for evaluation and learning purposes. It is not presented as a confidential production client system. External documentation and technical references used while preparing the report are listed in the references section.

Date: 4 June 2026

Place: Lovely Professional University

Signature of Student: Sanvi Gha

Declaration by Supervisors

This is to certify that the training report titled SmartCourier Delivery Management System has been prepared by the student as part of the Java Full Stack training program. The report reflects the student's understanding of full stack development, microservices architecture, API design, database connectivity, messaging, security, testing, and professional documentation.

The work is suitable for academic evaluation as a training project and should be read in that context. This certificate is subject to verification and signature by the authorized faculty or industry supervisor.

Name of External Supervisor: Capgemini

Training Certificate



Capgemini Technology Services India Limited
(Formerly known as IGATE Global Solutions Limited)
IT 1, IT 2, Airoli MIDC, Thane - Belapur Road,
Navi Mumbai 400708, Maharashtra, India.
Tel: +91 22 7144 4283 | Fax: +91 22 7141 2121
www.capgemini.com/in-en

Superset ID: 6752980

Letter of Intent ("LOI")

October 22, 2025

Dear Sanvi Ojha,

We are pleased to inform that your candidature has been shortlisted for the position of **Analyst/A4** with **Capgemini Technology Services India Limited** (hereinafter referred to as "Capgemini" or Company). You will be required to participate and complete the pre-onboarding training program assigned and applicable to you as may be communicated by the Company later. Please note that it is essential for you to participate, effectively leverage and successfully complete this program as a prerequisite prior to being onboarded as an employee with Capgemini.

We request you to carefully read and understand the Terms and Conditions of this Letter of Intent with Annexures hereto (hereinafter referred to as LOI).

- A Please note that your name mentioned in this LOI will be used to create your records in Capgemini & the same will be continued for all the communication & Company documentation purpose. In case you need a change in the name; please notify before commencement of training. Please note that no changes to the record can be made later in time. The name provided by you should match with the identification documents submitted to the Company, such as Aadhar Card, PAN card, Passport, etc.
- B We are proposing compensation package and benefits post-onboarding, the details of which are set forth in **Annexure 1** to this LOI.
- C Upon accepting this LOI, you will be required to submit a set of documents as mentioned in the **Annexure- 2** . Thereafter, you will be provided access to our pre-onboarding training program, as applicable. This will enable you to learn and master the concepts and skills required to be industry ready. The pre-onboarding training program can include physical classroom training/ self-paced e-learning/ hybrid model of training. The learning journey will be inclusive of assignments, assessments, hackathons/ competitions, and webinars as deemed appropriate by Capgemini.
- D The progress made by you in this learning journey would not only help you in getting onboarded but also help you to be trained for advanced skills relevant to your career at Capgemini. We also encourage you to learn beyond the prescribed course curriculum and acquire industry recognized certifications to accelerate your career in this competitive industry.
- E Pre-onboarding training Program and Terms & Conditions of the LOI
 - 1. Pre-onboarding Document Verification: Capgemini adheres to a strong document verification process. As a part of this process all the personal, educational and professional (if

Acknowledgement

I express my sincere gratitude to Lovely Professional University for providing the academic framework and opportunity to undertake training as part of the B.Tech Computer Science Engineering curriculum. I am thankful to Capgemini for providing a professional training environment where I could study full stack development, enterprise application structure, and project-style execution.

I also thank my mentors, trainers, faculty guides, peers, and team members for their support during the training period. Their feedback helped me connect React, Spring Boot, REST APIs, microservices, MySQL, RabbitMQ, JWT security, testing, and debugging into one coherent project. The SmartCourier project gave me a practical way to apply these concepts in a realistic courier-management domain.

Table of Contents

1. Chapter 1: Introduction of the Company
2. Chapter 2: Introduction of the Project Undertaken
3. Chapter 3: Microservices Architecture and System Design
4. Chapter 4: Implementation Details
5. Chapter 5: Work Done During Training
6. Chapter 6: Testing and Quality Assurance
7. Chapter 7: Conclusion and Future Scope
8. References

List of Figures

Figure Index

Figure No.	Title
Figure 1.1	Company organization structure in training context
Figure 3.1	SmartCourier microservices architecture
Figure 3.2	SmartCourier data flow diagram
Figure 3.3	SmartCourier use case diagram
Figure 3.4	Delivery booking and tracking sequence
Figure 4.1	SmartCourier entity relationship diagram

List of Tables

Table Index

Table No.	Title
Table 1.1	Capgemini service areas
Table 2.1	User roles and responsibilities
Table 2.2	Functional requirements
Table 2.3	Non-functional requirements
Table 3.1	System components and responsibilities
Table 4.1	Technology stack and rationale
Table 4.2	Core API endpoint summary
Table 5.1	Training activities and learning outcomes
Table 5.2	Weekly learning summary
Table 6.1	Sample test cases and results
Table 7.1	Future enhancements

List of Abbreviations

Abbreviation Index

Abbreviation	Full Form
API	Application Programming Interface
DTO	Data Transfer Object
DFD	Data Flow Diagram

ER	Entity Relationship
HTTP	Hypertext Transfer Protocol
JPA	Java Persistence API
JWT	JSON Web Token
LPU	Lovely Professional University
OJT	On-the-Job Training
REST	Representational State Transfer
SPA	Single Page Application
UI	User Interface

Chapter 1: Introduction of the Company

Capgemini is a global business and technology transformation partner that works with organizations on consulting, digital engineering, cloud, data, application development, cybersecurity, and managed service programs. During my training, I used Capgemini's learning environment to understand how professional software teams organize technical work, communicate design decisions, test APIs, and document systems for review.

The organization traces its origin to 1967, when Serge Kampf founded Sogeti in Grenoble, France. Over time, Capgemini expanded into a multinational technology group through consulting, technology services, engineering, outsourcing, and digital transformation capabilities. This background helped me understand why modern software projects must balance business requirements, maintainability, security, delivery discipline, and user experience.

1.1 Vision, Mission and Work Culture

The training program connected Capgemini's larger technology-service context with day-to-day engineering practices: writing maintainable code, documenting API contracts, testing changes, following security principles, and explaining design choices clearly. The work culture visible during training emphasized teamwork, regular feedback, process discipline, and continuous learning.

These values influenced the SmartCourier project. I treated it as more than a coding assignment: the system needed a defined domain, clear module responsibilities, understandable documentation, and verifiable behavior that an evaluator could trace from the user interface to the backend services.

1.2 Services and Departments

A technology services organization includes delivery teams, architecture groups, quality assurance teams, cloud and infrastructure teams, security teams, human resources, training departments, finance, sales, and client engagement functions. In my training context, the most relevant groups were training coordination, mentors, technical trainers, peer groups, and evaluation support.

Table 1.1: Capgemini Service Areas

Area	Description
Consulting	Business and technology advisory support for transformation programs.
Application Development	Design, development, modernization, and maintenance of software systems.
Cloud and Data	Cloud migration, data platforms, analytics, AI-enabled solutions, and managed services.
Engineering	Product engineering, intelligent industry, digital manufacturing, and platform engineering.
Cybersecurity	Security assessment, identity management, threat monitoring, and secure architecture practices.

1.3 Training Environment

My training ran from 18 December 2025 to 21 May 2026. The work involved guided learning, hands-on implementation, peer discussion, mentor feedback, debugging, API testing, and project documentation. I worked primarily in the Java Full Stack track.

The tools and technologies practiced during the project included React, Vite, React Router, Context API, Axios, Spring Boot, Spring Cloud, MySQL, RabbitMQ, Eureka, OpenFeign, JWT, Postman, Zipkin, JUnit, Mockito, and MockMvc. Since SmartCourier was a training project, I present it as a realistic learning implementation rather than as a live client delivery assignment.

Figure 1.1.1: Company Organization Structure in Training Context

Training Area	Role in Project Learning
Training Coordination	Defined the training timeline, learning checkpoints, and submission expectations.
Technical Mentors	Guided implementation concepts, debugging approach, and technology understanding.
Faculty/Evaluators	Reviewed academic presentation, report quality, and project explanation.
Peer Group	Supported discussion, testing practice, issue comparison, and viva preparation.

Chapter 2: Introduction of the Project Undertaken

SmartCourier Delivery Management System is a web-based courier and parcel delivery training project. It models user registration, authentication, parcel booking, delivery assignment, shipment status updates, tracking history, hub management, administrative monitoring, and report generation. I built and studied the system to understand how a full stack application behaves when responsibilities are separated across a frontend, API Gateway, backend services, databases, and a message queue.

The original requirement document referred to an Angular frontend with Spring Boot microservices. The implemented source code uses React with Vite, React Router, Context API, Axios, and reusable components. I have documented this difference because the architectural intent remains the same: modular UI pages, protected routes, role-based access, REST API integration, and a service-oriented backend.

2.1 Problem Statement

Courier operations require accurate booking, pickup scheduling, shipment status updates, hub monitoring, exception handling, customer visibility, and administrative control. When these functions are handled manually or placed inside tightly coupled modules, errors can occur in status visibility, reporting, traceability, and access control. SmartCourier addresses this at training-project scale by separating authentication, delivery, tracking, and administration into focused services.

2.2 Objectives

- Digitize the parcel booking and delivery lifecycle for a courier-management workflow.
- Provide separate customer and administrator flows with role-based access.
- Implement REST APIs for authentication, delivery management, tracking, hub management, reports, and dashboard data.
- Use JWT authentication so protected APIs can identify users and enforce roles.
- Practice database-per-service design using MySQL and JPA/Hibernate.
- Use RabbitMQ events to decouple delivery status updates from tracking history persistence.
- Test APIs using Postman and backend tests, including success, validation, missing-token, unauthorized, and not-found scenarios.

2.3 Scope and Boundaries

The implemented scope covers customer signup and login, delivery booking, customer delivery listing, tracking lookup, delivery status updates, admin dashboard, user and hub management, report APIs, service discovery, API Gateway routing, RabbitMQ-based tracking updates, and API testing. The project also includes architecture diagrams, Postman collections, test-case documentation, viva preparation notes, and technical walkthrough material.

The project does not include production deployment, payment processing, live GPS devices, shipment label printing, courier partner API integration, or organization-specific client business rules. These limits were deliberate because the project was designed for academic evaluation and full stack training.

Table 2.1: User Roles and Responsibilities

Role	Responsibilities
Customer	Register, log in, book delivery, view personal deliveries, track shipment status, and access customer pages only after authentication.
Administrator	View dashboard, manage deliveries, manage users, manage hubs, monitor reports, and update delivery status where required.
Delivery Agent (Extended Scope)	Receive assigned deliveries and update shipment status in a future extension.
Trainer/Evaluator	Review implementation, test APIs, inspect architecture, and evaluate project understanding.

Table 2.2: Functional Requirements

ID	Requirement	Module
FR-01	User registration, login, refresh, validation, logout, and role lookup.	Auth
FR-02	Create delivery with sender, receiver, source, destination, package weight, price, hub, and tracking details.	Delivery
FR-03	Assign delivery to an agent, cancel, ship, deliver, and update status.	Delivery
FR-04	Publish delivery status events and store tracking history.	Delivery/Tracking

FR-05	Fetch deliveries by customer, id, tracking number, status, agent, price threshold, and search parameters.	Delivery
FR-06	Manage logistics hubs with create, update, delete, and list operations.	Delivery
FR-07	Create, update, list, filter, count, and summarize reports.	Admin
FR-08	Show admin dashboard counts using service-to-service communication.	Admin

Table 2.3: Non-Functional Requirements

Attribute	Expected Behavior
Security	JWT-protected APIs and role-aware frontend routes restrict access.
Maintainability	Code is organized into controller, service, repository, DTO, entity, exception, and configuration layers.
Scalability	Microservices can be developed and scaled independently in the architecture model.
Reliability	RabbitMQ supports event-driven tracking updates and reduces direct coupling.
Usability	Frontend workflows guide users through login, booking, tracking, dashboard, and management pages.
Observability	Eureka and Zipkin support service visibility and request tracing during local evaluation.

Chapter 3: Microservices Architecture and System Design

SmartCourier follows a microservices-based architecture. The React frontend communicates with the Spring Cloud API Gateway, and the gateway routes requests to individual services. Each service owns a specific domain and uses its own persistence boundary. Eureka provides service discovery, RabbitMQ supports asynchronous events, OpenFeign supports synchronous service-to-service calls, and Zipkin helps trace requests during debugging.

SmartCourier Microservices Architecture

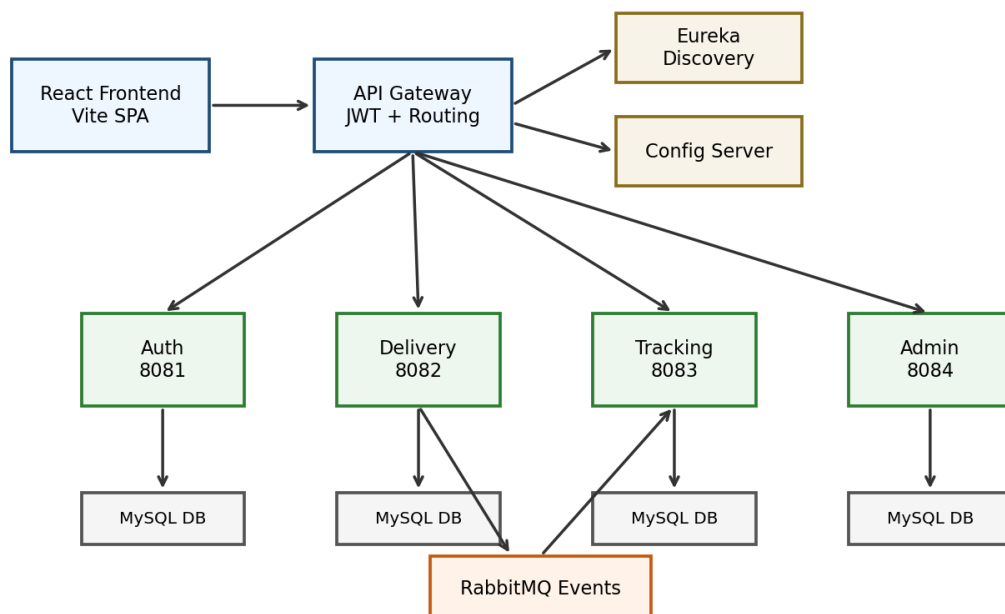


Figure 3.1: SmartCourier Microservices Architecture

Table 3.1: System Components and Responsibilities

Component	Responsibility	Learning Outcome
React Frontend	SPA screens, forms, protected routes, dashboards, and Axios API calls.	Component design and user flow.
API Gateway	Single external entry point for /gateway paths, token validation, and routing.	Centralized security and routing.
Eureka Server	Registers services and allows lookup by logical service name.	Dynamic service discovery.
Config Server	Centralizes configuration for services during local setup.	Configuration management.
Auth Service	Registration, login, JWT creation, user APIs, role lookup, validation, and logout.	Security and identity.
Delivery Service	Booking, pricing, hub management, assignment, status changes, and delivery search.	Core domain logic.
Tracking Service	Tracking events, latest status, range queries, counts, and event consumption.	Event-driven state history.
Admin Service	Reports, dashboard aggregation, delivery monitoring, and admin status updates.	Service aggregation and reports.
RabbitMQ	Message exchange and queues for delivery and user events.	Asynchronous communication.
MySQL	Relational storage for each service schema.	Persistence and JPA mapping.
Zipkin	Distributed tracing across gateway and services.	Debugging request paths.

3.1 Request Flow

1. The user interacts with the React frontend by logging in, booking a parcel, viewing deliveries, tracking a shipment, or opening an admin page.
2. Axios sends the request to the gateway base URL, `http://localhost:8080/gateway`.
3. The gateway validates JWT information for protected routes and forwards the request to the target service.
4. Eureka allows the gateway and Feign clients to locate services by logical service name instead of hardcoded service addresses.
5. The target service executes business logic through controller, service, repository, entity, DTO, mapper, and exception-handling layers.
6. When a delivery is created or its status changes, the Delivery Service publishes an event to RabbitMQ.
7. The Tracking Service consumes delivery status events and stores tracking history in its own database.
8. The Admin Service can fetch delivery data synchronously through OpenFeign to build dashboard summaries.

3.2 Data Flow and Use Cases

The data flow separates user input, authentication, delivery processing, tracking updates, and report generation. Customer use cases focus on signup, login, booking, viewing personal deliveries, and tracking. Administrator use cases focus on delivery monitoring, user and hub management, report generation, and dashboard visibility.

Data Flow Diagram - Level 1

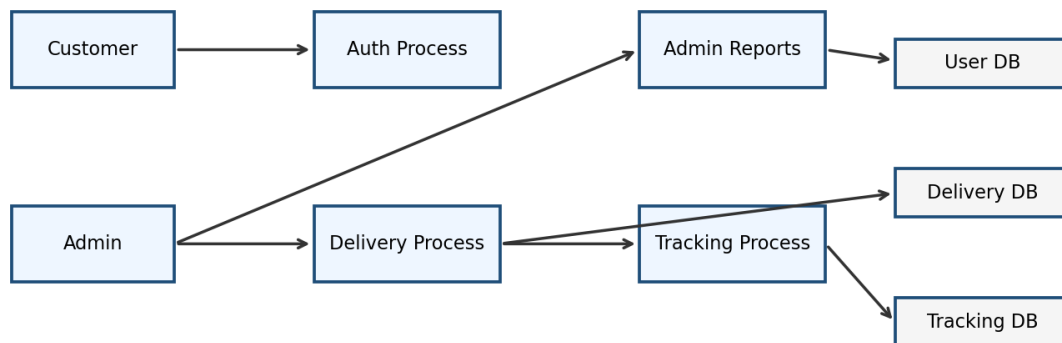


Figure 3.2: SmartCourier Data Flow Diagram

Use Case Diagram

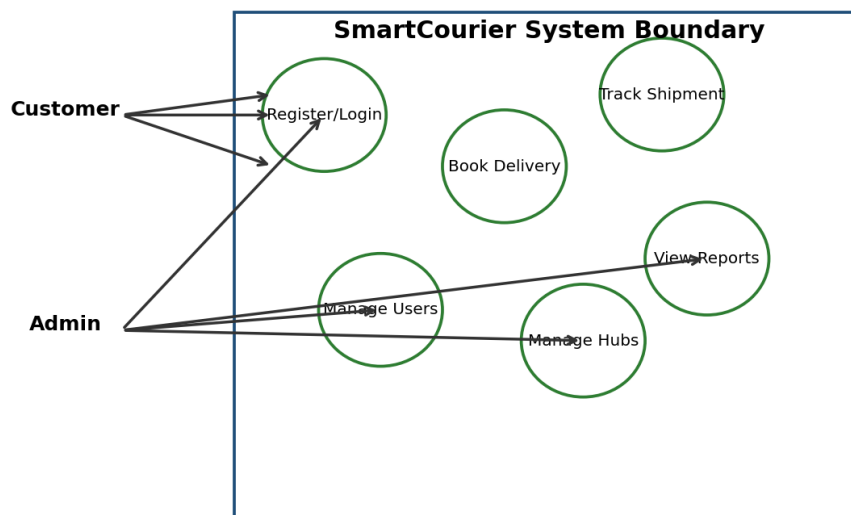


Figure 3.3: SmartCourier Use Case Diagram

3.3 Synchronous and Asynchronous Communication

Synchronous communication is used when the caller needs an immediate response. The frontend waits for login results, delivery details, tracking results, and dashboard data. The Admin Service also uses OpenFeign to fetch delivery information from the Delivery Service, while Eureka resolves the actual service instance.

Asynchronous communication is used when services should not block each other. The Delivery Service publishes events through RabbitMQ after booking creation or status updates. The Tracking Service listens to the configured queue and stores tracking records separately. This design keeps tracking history available without making the Delivery Service directly responsible for tracking persistence.

Sequence Diagram - Delivery Booking and Tracking Event

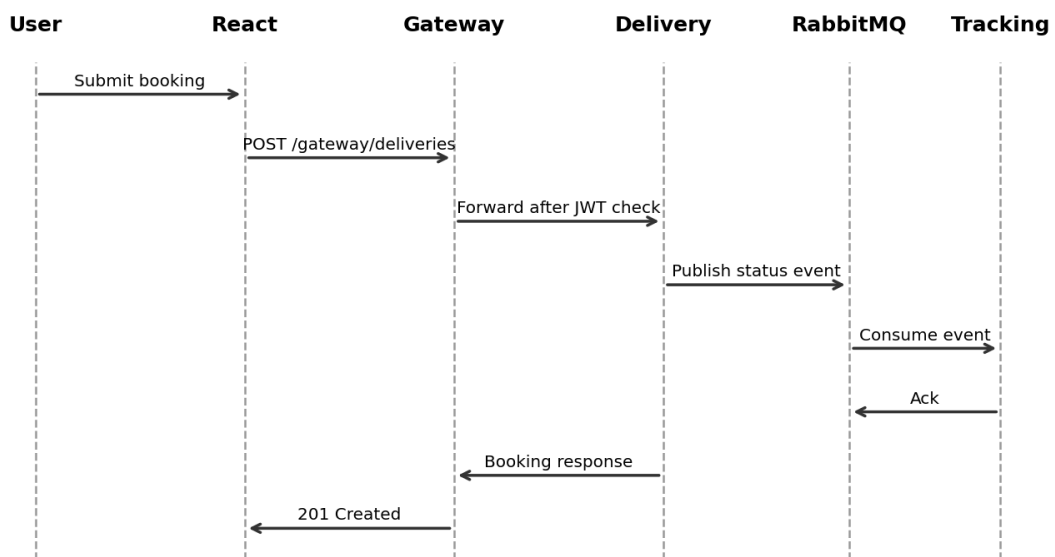


Figure 3.4: Delivery Booking and Tracking Sequence

Chapter 4: Implementation Details

4.1 Technology Choices

Table 4.1: Technology Stack and Rationale

Layer	Technology	Reason for Selection
Frontend	React, Vite, React Router, Context API, Axios	Fast SPA development, component-based UI, protected routing, global auth state, and HTTP integration.
Styling/UI	CSS/Tailwind-style utility conventions and reusable components	Consistent layout for landing, auth, customer, admin, and tracking pages.
Backend	Java 17+, Spring Boot 3.x, Spring Cloud	REST API development with gateway, discovery, Feign, security, and messaging support.
Security	Spring Security, JWT, BCrypt	Stateless authentication, role-aware access, and

		password hashing.
Persistence	MySQL, JPA/Hibernate, Flyway	Relational modeling, repository pattern, transactions, entity mapping, and repeatable migrations.
Messaging	RabbitMQ	Decouples delivery status changes from tracking persistence.
Service Discovery	Netflix Eureka	Allows services and gateway to use logical service names.
Observability	Zipkin/Micrometer tracing	Helps inspect request flow across multiple services.
Testing	JUnit, Mockito, MockMvc, Postman	Supports unit, controller, integration, and manual API validation.

4.2 Frontend Implementation

The frontend is implemented as a React single page application. Routing is centralized in `AppRoutes.jsx`. Public routes include the landing page, about page, login, and signup. Customer routes such as `/customer/dashboard`, `/customer/deliveries/new`, and `/customer/track/:trackingNumber` are protected with the CUSTOMER role. Admin routes such as `/admin/dashboard`, `/admin/deliveries`, `/admin/users`, and `/admin/hubs` are protected with the ADMIN role.

Authentication state is handled through `AuthContext`. The context keeps the logged-in user state and provides login, logout, and authentication-check behavior to navigation, public routes, and protected routes. This avoids prop drilling and keeps security state available wherever it is required.

API integration is handled through an `Axios` instance configured with the gateway base URL. The request interceptor attaches the JWT token to outgoing requests, while the response logic can redirect the user when authorization fails. This keeps token handling centralized instead of repeating header logic inside every page.

4.3 Backend Implementation

The backend is divided into services. Each service follows a layered Spring Boot structure: controller classes expose REST endpoints, service classes contain business logic, repository interfaces handle database access, entities map to database tables, DTOs shape request and response payloads, and exception handlers return consistent error responses.

The Auth Service implements signup, login, refresh, token validation, current-user lookup, role lookup, user listing, user update, role update, deletion, and logout. The User entity stores username, email, phone number, encrypted password, role, active status, created timestamp, and updated timestamp. `BCrypt` is used so raw passwords are not stored.

The Delivery Service is the main business service. It stores delivery records with tracking number, customer username, sender and receiver details, source, destination, status, assigned agent, current hub, price, weight, and timestamps. It also manages hubs with name, code,

city, state, address, contact number, and audit timestamps. Pricing is implemented through a Strategy Pattern in which PricingStrategyFactory selects standard or express pricing logic based on delivery characteristics.

The Tracking Service stores tracking events with delivery id, status, location, description, and timestamp. It exposes APIs for creating tracking events, retrieving all events for a delivery, latest status, status/location filters, date-range filtering, counts, existence checks, deletion, and health checks. It also consumes RabbitMQ messages so delivery status changes become tracking history.

The Admin Service manages reports and dashboard-related operations. Reports contain report type, report date, from/to range, report data, and generated-by information. The service exposes report CRUD, filtering, count, latest report, type summaries, dashboard, delivery monitoring, and admin delivery status update APIs.

4.4 Database Design

SmartCourier follows database-per-service ownership. Auth data is owned by the Auth Service, delivery and hub data by the Delivery Service, tracking history by the Tracking Service, and report data by the Admin Service. This prevents one service from directly modifying another service's tables and keeps domain boundaries clear.

ER Diagram - Logical Training Project Schema

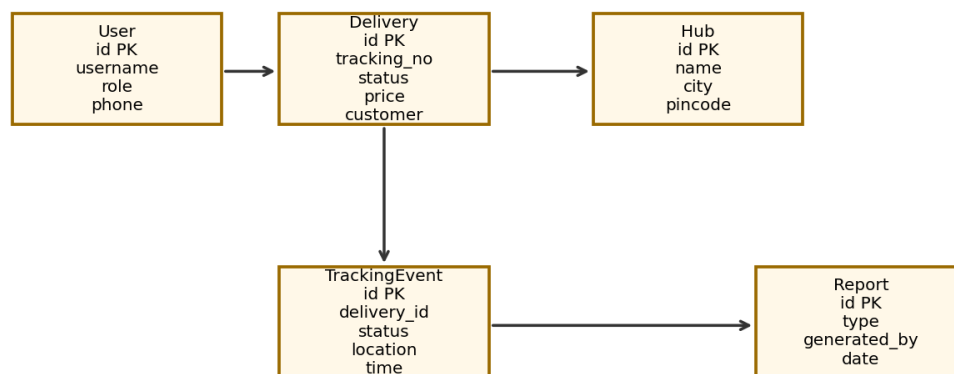


Figure 4.1: SmartCourier Entity Relationship Diagram

4.5 API Design

The APIs are grouped by service and exposed through the gateway under /gateway. Direct service URLs are also useful during debugging: Auth runs on port 8081, Delivery on 8082, Tracking on 8083, Admin on 8084, and the gateway on 8080. The test documentation identifies 62 implemented endpoints across the four business services.

Table 4.2: Core API Endpoint Summary

Service	Endpoint Group	Purpose
Auth	POST /auth/signup, POST /auth/login, POST /auth/refresh	Registration, login, and token refresh.
Auth	GET /auth/validate, /role, /me, /users/*	Token validation, current user, role lookup, and user management.
Delivery	GET /deliveries/my, POST /deliveries, GET /deliveries/{id}	Customer delivery listing, booking, and detail retrieval.
Delivery	PUT /deliveries/{id}/status, /assign, /cancel, /ship, /deliver	Lifecycle and assignment operations.
Delivery	GET /deliveries/tracking/{trackingNumber}, /summary/status, /search	Tracking lookup, status summaries, and filtering.
Delivery	/deliveries/hubs	Hub create, update, delete, detail, and list operations.
Tracking	POST /tracking, GET /tracking/{deliveryId}, /latest, /range	Tracking event creation and retrieval.
Tracking	/status/{status}, /location/{location}, /count, /exists	Tracking filters and status checks.
Admin	/admin/reports, /admin/reports/latest, /summary/type	Report management and summaries.
Admin	/admin/dashboard, /admin/deliveries, /admin/deliveries/{id}/status	Dashboard, delivery monitoring, and admin status updates.

4.6 Security Implementation

Security is implemented through JWT-based authentication. After a successful login, the Auth Service returns a token. The frontend stores the token and sends it in the Authorization: Bearer <token> header for protected API calls. The API Gateway validates the token before forwarding secured requests. Role-based logic is applied in protected frontend routes and backend access checks.

For downstream services, the gateway can propagate identity details through headers such as username or role. The Delivery Service uses customer identity to return only the current user's deliveries through /deliveries/my and related endpoints. This avoids relying on user-submitted customer identifiers for protected customer data.

Chapter 5: Work Done During Training

During the training, I worked through SmartCourier as an end-to-end learning exercise. I studied the requirement document, understood the courier domain, set up the frontend and backend, connected services through gateway routes, practiced database persistence, implemented and tested APIs, and prepared documentation for evaluation.

Table 5.1: Training Activities and Learning Outcomes

Activity	Work Performed	Learning Outcome
Orientation	Understood training objectives, tools, project expectations, and the full stack roadmap.	Prepared environment and learning notes.
React Practice	Worked on pages, forms, routing, protected routes, dashboards, and reusable components.	Improved UI component and workflow understanding.

Spring Boot Practice	Created controllers, services, repositories, DTOs, entities, exception handlers, and configuration classes.	Understood layered backend design.
Database Connectivity	Configured MySQL, JPA entities, repositories, and service-level persistence.	Practiced schema design and repository pattern.
Authentication	Implemented and tested signup, login, JWT token usage, protected APIs, and role checks.	Understood stateless security workflow.
RabbitMQ	Published delivery status events and consumed tracking updates.	Learned asynchronous service communication.
Testing	Used Postman, unit tests, controller tests, validation checks, and integration checklists.	Improved debugging and QA discipline.
Documentation	Prepared report, architecture notes, viva question bank, walkthrough, and test-case documentation.	Improved technical communication.

5.1 Delivery Service Deep Dive

The Delivery Service was the most important module because it holds the core courier workflow. A booking request enters through the controller, is validated through DTO constraints, passes into the service layer, calculates pricing through the pricing strategy, saves the record through JPA, and publishes an event for tracking. This module helped me understand how business logic, validation, persistence, and messaging fit together.

Important decisions in this module include using an enum for delivery status instead of a free-form string, generating a unique tracking number, storing the authenticated customer username, linking the current hub through a JPA relationship, and using `@Transactional` service methods so related changes either complete together or fail safely.

5.2 Challenges Faced and Solutions Implemented

- Challenge: Understanding request flow across frontend, gateway, services, database, and queue. Solution: traced one workflow at a time from UI action to API, service method, database record, RabbitMQ message, and tracking result.
- Challenge: Handling the React-versus-Angular difference between the specification and implementation. Solution: preserved the original requirement intent while documenting the actual React implementation accurately.
- Challenge: Debugging protected requests. Solution: checked token creation, browser storage, Axios headers, gateway validation, role checks, and downstream service headers in sequence.
- Challenge: Avoiding tightly coupled services. Solution: used RabbitMQ for status events and OpenFeign only where immediate dashboard data was required.
- Challenge: Validating APIs beyond happy paths. Solution: created Postman and test-case coverage for missing tokens, invalid bodies, invalid ids, duplicate users, unauthorized roles, and unavailable downstream services.
- Challenge: Explaining microservices clearly for viva. Solution: prepared architecture diagrams, walkthrough notes, endpoint summaries, and module-level explanations.

5.3 Weekly Learning Summary

Table 5.2: Weekly Learning Summary

Week	Learning Focus	Work Summary
Week 1	Java refresh and OOP	Practiced core Java concepts and prepared for backend work.
Week 2	Spring Boot basics	Created basic REST controllers and understood application structure.
Week 3	REST API practice	Worked with HTTP methods, JSON payloads, status codes, and Postman.
Week 4	React components	Built and reviewed reusable UI components.
Week 5	Routing and Context API	Implemented protected routes and global authentication state.
Week 6	MySQL and JPA	Mapped entities, repositories, and database operations.
Week 7	JWT security	Studied login, token generation, token validation, and role checks.
Week 8	Microservices and Eureka	Registered services and understood discovery flow.
Week 9	RabbitMQ messaging	Published and consumed events for tracking updates.
Week 10	OpenFeign and admin dashboard	Practiced service-to-service communication and fallback thinking.
Week 11	Postman testing	Prepared API requests for integrated testing.
Week 12	Debugging and exception handling	Handled validation and not-found scenarios more consistently.
Week 13	Frontend integration	Connected UI flows with gateway APIs.
Week 14	Unit testing	Practiced JUnit, Mockito, and controller tests.
Week 15	Documentation	Created report sections and architecture notes.
Week 16	Viva preparation	Prepared question bank and walkthrough material.
Week 17	Final review	Reviewed source, APIs, diagrams, and testing evidence.
Week 18	Future scope study	Documented cloud, GPS, notifications, and route optimization improvements.
Week 19	Report drafting	Organized chapters, tables, figures, and references.
Week 20	Submission readiness	Reviewed formatting, grammar, duplication, and evaluator flow.

Chapter 6: Testing and Quality Assurance

Testing focused on verifying individual services, API contracts, secured request behavior, and cross-service workflows. Since SmartCourier is a training project, I used a practical combination of automated backend tests, controller tests, Postman collections, validation checks, and manual UI workflow verification.

6.1 Testing Strategy

Unit tests were used for service-layer behavior where dependencies could be mocked. Mockito helped isolate logic that depended on repositories, mappers, pricing strategies, RabbitMQ publishers, or Feign clients. Controller tests checked whether REST endpoints accepted input and returned expected HTTP responses.

Postman testing was used for integrated API verification. The collection uses the gateway base URL and variables for access token, refresh token, usernames, password, phone number, delivery id, tracking number, hub id, report id, and assigned agent. Login and refresh flows can save tokens for later protected requests.

Integration-focused checks covered Auth plus Gateway, Delivery plus RabbitMQ plus Tracking, Admin plus Feign plus Delivery, Eureka registration, Config Server access, and Zipkin traces. These checks helped me understand failures caused by configuration, service discovery, message queues, token headers, or database state.

Table 6.1: Sample Test Cases and Results

ID	Scenario	Expected Result	Status
TC-01	Signup with valid username, phone number, and password.	User registered successfully.	Pass
TC-02	Login with valid credentials.	JWT access token returned.	Pass
TC-03	Access protected delivery API without token.	401 Unauthorized.	Pass
TC-04	Create delivery with valid request.	Delivery created with CREATED status and tracking number.	Pass
TC-05	Update delivery status to SHIPPED.	Status updated and RabbitMQ event published.	Pass
TC-06	Consume delivery status event.	Tracking event stored by Tracking Service.	Pass
TC-07	Customer opens admin route.	403 Forbidden or frontend redirect.	Pass
TC-08	Admin dashboard fetches counts.	Dashboard values returned or safe fallback shown.	Pass
TC-09	Invalid delivery id lookup.	404 Not Found or handled error response.	Pass
TC-10	Eureka dashboard after startup.	Services shown as UP.	Pass

6.2 Evaluation Demo Flow

9. Show Eureka with gateway, config server, and business services running.
10. Open Config Server URL to confirm service configuration is available.
11. Sign up or log in through the Auth API.
12. Use the returned JWT on gateway delivery APIs.
13. Create a delivery and note the tracking number.
14. Update delivery status and observe RabbitMQ queue activity.
15. Fetch tracking history to confirm the event was stored.
16. Open the admin dashboard to verify dashboard counts.
17. Show Zipkin traces for a gateway request and downstream service call.
18. Show Swagger or Postman requests for selected services.

6.3 Results Achieved

The final training implementation demonstrates a working full stack courier-management flow. Customers can authenticate, book deliveries, view their deliveries, and track status. Administrators can access protected management views and dashboard/report APIs. The

backend demonstrates 62 service endpoints across Auth, Delivery, Tracking, and Admin services. RabbitMQ-based events update tracking records, while OpenFeign supports administrative aggregation.

The project also produced supporting artifacts: architecture diagrams, ER diagram, sequence diagram, data-flow diagram, use-case diagram, Postman collection, test-case documentation, viva question bank, technical concept explanations, and project walkthrough notes. These outputs made the project easier to evaluate and explain.

Chapter 7: Conclusion and Future Scope

The training helped me connect academic knowledge with practical full stack software development. Through SmartCourier, I understood how a courier-management system can be divided into clear responsibilities: the frontend handles user workflows, the gateway handles routing and token checks, each service owns its domain logic, repositories manage persistence, and messaging supports asynchronous updates.

The project improved my confidence in React, Spring Boot, REST APIs, JWT security, MySQL, JPA/Hibernate, RabbitMQ, Eureka, OpenFeign, Postman, Zipkin, and unit testing concepts. More importantly, it helped me explain request flow, justify architecture choices, test APIs systematically, and debug integration issues across multiple layers.

7.1 Key Learnings

- A full stack feature must be understood from the user interface through the gateway, service, database, and response.
- JWT security is not only a login feature; it affects routing, API headers, role checks, protected screens, and error handling.
- Microservices improve separation of responsibilities but introduce configuration, discovery, messaging, and debugging complexity.
- RabbitMQ is useful when one service needs to notify another without waiting for that service to complete work immediately.
- Testing must include success cases, invalid input, missing authorization, role restrictions, not-found records, and service integration scenarios.
- Good documentation makes technical work easier to evaluate, maintain, and explain in viva.

7.2 Future Scope

Table 7.1: Future Enhancements

Enhancement	Description
AI-Based Route Optimization	Recommend efficient routes using distance, delivery priority, hub capacity, traffic data, and historical patterns.
Real-Time GPS Tracking	Show live parcel or delivery-agent location using map integration and privacy-aware location updates.
Payment Integration	Support online payment, invoices, refunds, and secure transaction records.
Notification System	Send email, SMS, or push notifications for booking confirmation, status

	changes, delays, and delivery proof.
Shipment Labels and Documents	Generate labels, invoices, proof-of-delivery documents, and upload supporting files.
Cloud Deployment	Containerize services and deploy with managed databases, broker, secrets management, logging, and monitoring.
CI/CD and Quality Gates	Automate build, test, code quality checks, and deployment pipelines.
Centralized Logging and Metrics	Add dashboards for service health, latency, error rates, queue depth, and trace analysis.
API Rate Limiting	Protect public endpoints from abuse and improve gateway-level security.

7.3 Final Conclusion

SmartCourier remained within the scope of an academic and training project, but it successfully modeled realistic logistics workflows and modern application design patterns. The project demonstrates how authentication, delivery booking, tracking, administration, database design, microservices communication, security, testing, and documentation can be combined into one coherent system. It strengthened my ability to build, analyze, and explain enterprise-style full stack applications.

References

1. Capgemini. About us. <https://www.capgemini.com/us-en/about-us/>
2. Capgemini. What we do: services and solutions. <https://www.capgemini.com/us-en/about-us/who-we-are/what-we-do/>
3. Capgemini. Who we are. <https://www.capgemini.com/gb-en/about-us/who-we-are/>
4. Spring Boot Reference Documentation. <https://docs.spring.io/spring-boot/docs/current/reference/html/>
5. Spring Framework Documentation. <https://docs.spring.io/spring-framework/reference/>
6. Spring Cloud Gateway Documentation. <https://docs.spring.io/spring-cloud-gateway/reference/>
7. Spring Cloud Netflix Eureka Documentation. <https://docs.spring.io/spring-cloud-netflix/reference/>
8. Spring Data JPA Documentation. <https://docs.spring.io/spring-data/jpa/reference/>
9. React Documentation. <https://react.dev/>
10. React Router Documentation. <https://reactrouter.com/>
11. Axios Documentation. <https://axios-http.com/docs/intro>
12. Oracle Java Documentation. <https://docs.oracle.com/en/java/>
13. RabbitMQ Documentation. <https://www.rabbitmq.com/docs>
14. MySQL Documentation. <https://dev.mysql.com/doc/>
15. JSON Web Token Introduction. <https://jwt.io/introduction>
16. OpenAPI Specification. <https://spec.openapis.org/oas/latest.html>
17. Postman Learning Center. <https://learning.postman.com/docs/>
18. Hibernate ORM Documentation. <https://hibernate.org/orm/documentation/>